

Flow-Constrained Counterfactual Graph Augmentation for Robust Transaction Fraud Detection

Anonymous Author(s)

ABSTRACT

Fraud and anti-money laundering (AML) detection on transaction graphs is often limited by emerging typologies, scarce positive labels, and distribution shift. Data augmentation is a natural response, but standard feature-space or unconstrained adversarial augmentation can create examples that are hard for a detector because they violate financial structure: inconsistent amount movement, invalid categorical fields, impossible timestamps, or unrealistic account-history behavior. We study flow-constrained counterfactual augmentation, a validity-aware training procedure that generates local hard transaction edits and projects them toward auditable flow-balance, temporal, categorical, and account-profile constraints. The projection operates locally on candidate transaction records and is not a global financial-flow simulator.

The method first trains a detector, proposes candidate edits around observed positive transactions, scores projected candidates by detector loss, and keeps those that satisfy profile-plausibility gates. The objective is not to synthesize new fraud episodes from scratch. It is to test whether controlled, structurally-plausible hard support can improve detection when a future AML typology is poorly represented or absent from training. We evaluate this question primarily through AMLNet held-out layering, with additional checks on profile-rich and external transaction benchmarks. Baselines separate three possibilities: ordinary feature-space augmentation, post-hoc repaired augmentation, and valid but non-hard random augmentation.

On held-out layering, feasibility-projected counterfactuals improve boosted-tree detectors over no augmentation, repaired standard baselines, and random feasible augmentation, and the improvement persists under alert-budget, few-shot, temporal, and new counterparty-pair slices. The method does not transfer to all typologies or to graph neural detectors. Validity is therefore not a universal performance guarantee: it is a necessary audit condition whose predictive value depends on whether the generated support matches the geometry of the target typology.

CCS CONCEPTS

• **Information systems** → **Data mining; Fraud detection;** • **Computing methodologies** → *Neural networks.*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '26, November 7–11, 2026, Rome, Italy

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

KEYWORDS

fraud detection, anti-money laundering, counterfactual augmentation, transaction graphs, flow constraints, typology transfer, distribution shift, data augmentation

ACM Reference Format:

Anonymous Author(s). 2026. Flow-Constrained Counterfactual Graph Augmentation for Robust Transaction Fraud Detection. In *Proceedings of 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*. ACM, New York, NY, USA, 10 pages.

1 INTRODUCTION

Transaction fraud and anti-money-laundering (AML) detection sit at the intersection of three unforgiving constraints: positives are rare, labels arrive late, and the target behavior keeps changing. By the time investigators have confirmed enough cases of an emerging laundering typology to train an ordinary supervised model, the typology has often already migrated. Data augmentation is therefore a natural lever. If a detector has only a small set of confirmed cases of a new pattern, well-chosen synthetic positives could in principle help it learn a decision boundary that transfers to the next batch of real ones.

The difficulty is that transactions are not arbitrary feature vectors. A generated transaction implicitly asserts that money could have moved between two accounts in a way that respects local flow balance, the categorical structure of payment types, the temporal ordering of the sender's and receiver's histories, and the behavioral profile of the entities involved. Off-the-shelf augmentation methods encode none of this. Feature noise produces negative transaction counts and fractional one-hot categories; SMOTE [3] and mixup [39] leave the categorical manifold; and unconstrained adversarial perturbations [13, 21] are hard for the detector *because* they are financially impossible. The consequence is a quiet form of train-test mismatch: predictive scores improve by teaching the detector to react to patterns that would never appear in a deployed transaction stream.

We define this property as the *validity gap* between augmented examples and the support of real transaction records, and we study whether closing it is both feasible and useful. Our approach starts from confirmed positive seeds, proposes detector-challenging local edits, projects them onto a transaction-feasibility manifold defined by local flow balance, categorical integrity, account-profile bounds, and temporal feasibility, and filters them by an account-profile plausibility score. The pipeline does not simulate entire laundering episodes or enforce a global flow invariant across the whole graph; it is a per-edge procedure for generating auditable local counterfactuals.

We evaluate this procedure on a held-out typology transfer protocol in which all positives of a target typology are removed from training and detection is evaluated only on future positives of that typology. On held-out layering, flow-constrained counterfactuals improve both ranking and operational alert-budget metrics over no

augmentation, repaired SMOTE/mixup, and random feasible augmentation, with paired bootstrap intervals that exclude zero across ten seeds. Gains persist after a small number of target-typology labels arrive, across temporal drift, and on previously unseen counterparty pairs. They do *not* persist on structuring, integration, or graph neural detectors. A mechanism analysis attributes the typology gap to amount-regime mismatch between generated and target supports: validity is necessary for financial augmentation but not sufficient, and the generated examples must also lie in the semantic region that defines the target typology.

Contributions.

- We formalize a four-part *financial-validity audit* for transaction augmentation—local flow balance, categorical integrity, account-profile consistency, and temporal feasibility—and show that common augmentations fail multiple parts of this audit simultaneously, while repaired and random-feasible variants pass it but contribute little decision-boundary pressure.
- We introduce FC-CGA, a validity-aware augmentation pipeline that combines detector-hard candidate generation, a closed-form feasibility projection operator, account-profile plausibility gating, and an optional typology curriculum. The projection adds no trainable parameters to the downstream detector.
- We provide a typology-transfer evaluation that separates predictive and validity conclusions. Across ten seeds and paired bootstrap intervals, projected counterfactuals significantly improve held-out layering AUPRC, low-FPR recall, and top-alert precision, while repaired feature-space baselines do not. A mechanism analysis of generated-vs. target support geometry explains both the positive layering result and the failures on structuring, integration, and graph neural detectors.

The remainder of the paper formalizes the problem (§3), describes the pipeline and feasibility operator (§4), details the experimental protocol and reports the main and stress-test results (§5–§6), and closes with constraint ablations and mechanism analysis (§7).

2 RELATED WORK

Transaction graph fraud detection. Early work cast laundering detection as node or edge classification on Bitcoin transaction graphs with graph convolutional networks [33, 34]. Subsequent methods added temporal dynamics with EvolveGCN [23] and spatial-temporal attention [5], tackled camouflage in fraudulent behavior [10], and addressed class imbalance via neighbor sampling [18]. Surveys catalogue these methods [25]. This line of work largely treats label scarcity as a training-objective problem, leaving the question of *what to add to the training set* underexplored.

AML simulation and benchmarks. The Elliptic dataset [34] and its successor Elliptic++ [11] provide Bitcoin graph benchmarks for supervised laundering classification. The IBM AMLSim simulator [28, 29] and the multi-bank simulator of [1] generate synthetic transaction flows with embedded typologies—layering, structuring, fan-in/fan-out—together with ground-truth typology labels. We use an AMLSim-derived benchmark (which we refer to as AMLNet in the rest of the paper) for our typology-transfer protocol because it provides per-positive typology labels that make held-out typology transfer well defined.

Data augmentation for class imbalance. SMOTE [3] interpolates minority-class examples in feature space; mixup [39] extends interpolation across class boundaries. CutMix [38] and AutoAugment [6] extend image-space augmentation strategies. GraphSMOTE [41], GraphMixup [35], GraphENS [24], and IMGAGN [27] adapt these ideas to imbalanced node classification on graphs. FLAG [17] applies feature-space adversarial perturbations during GNN training, and DeepSMOTE [7] combines deep generation with SMOTE. Graph contrastive augmentations [15, 37, 42] focus on self-supervised pre-training. Each of these methods treats augmentation as a purely predictive concern: there is no notion of whether a generated sample could exist as a record in the underlying domain.

Counterfactual methods. Counterfactual reasoning has primarily been studied as a post-hoc *explanation* tool that searches for the minimal feature change flipping a classifier’s prediction [31, 32]. FACE [26] adds feasibility constraints to that search, CLEAR [20] uses generative models for causal counterfactuals, and CF-GNNExplainer [19] modifies graphs to explain GNN decisions. Closer to our setting, [40] uses counterfactual graph perturbations for link-prediction training, and [22] argues for causal constraint preservation in counterfactual generation. None of this work targets transaction-graph fraud, and none enforces financial-domain constraints—local flow balance, payment-type integrity, account-profile bounds, and temporal feasibility—during candidate generation.

Distribution shift and emerging-typology transfer. Temporal concept drift in fraud detection is well documented [8, 12], and graph-based dynamic adaptation has been explored under temporal GNNs [2, 36]. The held-out typology transfer protocol we adopt is orthogonal to temporal drift: it tests whether a detector trained without *any* positives of a target typology can recover detection on future positives of that typology when paired with a validity-aware augmentation procedure.

3 PROBLEM FORMULATION

Transaction graph. A financial transaction graph is a directed multigraph $G = (V, E, X)$ where V is a set of accounts (nodes), E is a set of transaction edges, and $X = \{x_e\}_{e \in E}$ are edge feature vectors. Each transaction $e = (u, v, t)$ represents a fund transfer from account u to account v at time t , with feature vector $x_e \in \mathbb{R}^d$ encoding the log-amount, categorical payment type (one-hot), normalised timestamp, account-history aggregates, and local neighbourhood statistics.

Fraud detection task. A binary detector $f_\theta : X \rightarrow [0, 1]$ assigns a fraud probability to each transaction edge, trained on labelled positives (fraud) and negatives (legitimate). Labels are highly imbalanced: the positive rate in AMLNet is approximately 0.16% and in TransXion approximately 0.15%. We use AUPRC as the primary metric, as it is appropriate for severely imbalanced detection tasks [9].

Financial validity constraints. A generated transaction example $x' \in \mathbb{R}^d$ is financially valid if and only if all of the following hold:

- (1) *Local flow balance:* The ratio of total outflow to total inflow for account v satisfies $\left| \frac{\sum_{e \in \text{out}(v)} a_e}{\sum_{e \in \text{in}(v)} a_e} - 1 \right| \leq \delta$ for tolerance $\delta > 0$, where a_e is the transaction amount.
- (2) *Temporal feasibility:* For each account v with transaction history $\{t_1, \dots, t_k\}$, a new transaction time t' satisfies $t_{\min} \leq$

$t' \leq t_{\max}$ and maintains chronological ordering within the account window.

- (3) *Categorical validity*: All one-hot payment-type indicators $c_e \in \{0, 1\}^K$ remain in $\{0, 1\}^K$ with $\|c_e\|_1 = 1$.
- (4) *Profile consistency*: Augmented account-history features $h' \in \mathbb{R}^P$ satisfy $h'_j \in [\mu_{v,j} - k\sigma_{v,j}, \mu_{v,j} + k\sigma_{v,j}]$ for each dimension j , where $\mu_{v,j}$ and $\sigma_{v,j}$ are the observed mean and standard deviation of account v in the training period, and $k \geq 1$ is a configurable width.

The first constraint is local to the edited account context: we do not propagate edits through downstream accounts or synthesize a balanced multi-edge episode. The augmentation procedure is therefore an auditable local edit rather than an end-to-end financial simulator.

Augmentation problem. Given a detector f_θ , a set of confirmed positive transactions $P \subset E$, and a validity oracle $\mathcal{V}$, generate a counterfactual set C such that: (a) each $x' \in C$ is financially valid ($\mathcal{V}(x') = 1$); (b) the detector assigns high loss to x' (high hardness); (c) each x' is typology-consistent with the target fraud pattern; and (d) adding C to the training set improves generalisation to held-out positive distributions. We denote the augmented training set $T' = T \cup C$ with $|C| = \lfloor \rho|P| \rfloor$ for augmentation ratio ρ .

4 METHOD

4.1 Overview

Figure 1 shows the FC-CGA pipeline. The design separates three roles that are usually conflated in augmentation papers. *Candidate generation* makes examples hard for the current detector; *feasibility projection* repairs them into auditable transaction records; and a *separate validity audit* reports artifact rates and profile drift without folding them into the training signal. This decoupling is what makes it possible to identify methods that improve scores by exploiting invalid artifacts (such as unprojected adversarial perturbations) and distinguish them from methods that improve scores through valid hard support. The core algorithmic components are the closed-form projection operator (Algorithm 1) and the generation procedure (Algorithm 2).

4.2 Perturbation

Starting from a seed positive transaction $x \in P$, a candidate $\tilde{x}$ is generated by perturbing three complementary feature groups:

- (1) *Amount perturbation*: $\tilde{a} = a \cdot (1 + \epsilon_a)$ where $\epsilon_a \sim \mathcal{U}(-\sigma_{\text{amt}}, \sigma_{\text{amt}})$ scaled to the empirical amount spread of positive examples.
- (2) *History perturbation*: Aggregated features (e.g. rolling transaction count, inflow sum, neighbour degree) are perturbed within a configurable profile-width multiplier.
- (3) *Boundary-mode perturbation*: For hard and typology-aware variants, feature-direction gradients from the trained detector f_θ are used to move the candidate towards the positive-class region, maximising detector loss.

For the typology-projected variant, perturbation magnitudes are weighted by feature importance under that typology: layering examples emphasise multi-hop-flow features and chain-length statistics; structuring emphasises repeated-small-amount patterns. These weights are fixed from analyst-specified feature groups and

Algorithm 1 FeasibilityProject

Require: Perturbed candidate $\tilde{x}$; account stats $\{\mu_v, \sigma_v\}$; profile width k

Ensure: Validity-compliant projected example x'

- 1: $a' \leftarrow \text{LocalFlowBalance}(\tilde{a})$ ▷ Eq. 1
- 2: $c' \leftarrow \text{ArgmaxOneHot}(\tilde{c})$ ▷ Eq. 2
- 3: $h' \leftarrow \text{clip}(\tilde{h}; \mu_v - k\sigma_v, \mu_v + k\sigma_v)$ ▷ Eq. 3
- 4: $h' \leftarrow \max(h', 0)$ ▷ Non-negativity
- 5: $t' \leftarrow \text{clip}(\tilde{t}; t_{\min}^{(v)}, t_{\max}^{(v)})$ ▷ Eq. 4
- 6: **return** $x' \leftarrow (a', c', h', t')$

training-time metadata, not estimated from held-out target positives. This makes the typology-projected row an analyst-prior setting rather than a zero-knowledge emerging-typology setting. We therefore also report hard and curriculum projected variants that do not rely on target-typology weights.

4.3 Feasibility Projection

The feasibility projection maps an arbitrary perturbed candidate $\tilde{x} \in \mathbb{R}^d$ to the nearest point in the validity space, solving four sub-problems in sequence.

Local flow-balance projection. Let $\tilde{a}_e$ be the perturbed amount for transaction e originating from account v . Define the row-sum normalisation:

$$a'_e = \tilde{a}_e \cdot \frac{\sum_{e' \in \text{in}(v)} a_{e'}}{\sum_{e' \in \text{out}(v)} \tilde{a}_{e'}}, \quad (1)$$

which ensures $\sum_{\text{out}(v)} a'_e = \sum_{\text{in}(v)} a_{e'}$ within the edited local account context. This removes the audited amount residual for the generated transaction, but it is not a global conservation operator over all graph edges.

Categorical repair. Each categorical one-hot block is repaired by setting the maximum-valued dimension to one and all others to zero:

$$c'_k = 1[k = \arg \max_j \tilde{c}_j], \quad k = 1, \dots, K. \quad (2)$$

Profile constraint projection. Each account-history dimension j is clipped:

$$h'_j = \text{clip}(\tilde{h}_j; \mu_{v,j} - k\sigma_{v,j}, \mu_{v,j} + k\sigma_{v,j}). \quad (3)$$

Negative-valued features (e.g. transaction counts) are additionally floored at zero.

Temporal feasibility. The perturbed timestamp is clamped to the valid window:

$$t' = \text{clip}(\tilde{t}; t_{\min}^{(v)}, t_{\max}^{(v)}). \quad (4)$$

The composed projection is $\text{Proj}(\tilde{x}) = (a', c', h', t')$. By construction, $\text{Proj}(\tilde{x})$ satisfies all four audited local validity constraints simultaneously.

4.4 Plausibility Gating

The plausibility score of a projected example x' is defined as:

$$S_p(x') = \exp(-\lambda \cdot \text{drift}(x', \mu_v)), \quad (5)$$

where $\text{drift}(x', \mu_v) = \|x'_h - \mu_v\|_2 / \|\sigma_v\|_2$ is the normalised profile deviation, and $\lambda > 0$ is a decay parameter. A candidate is accepted if $S_p(x') > \tau$, i.e. if its normalised profile deviation is below

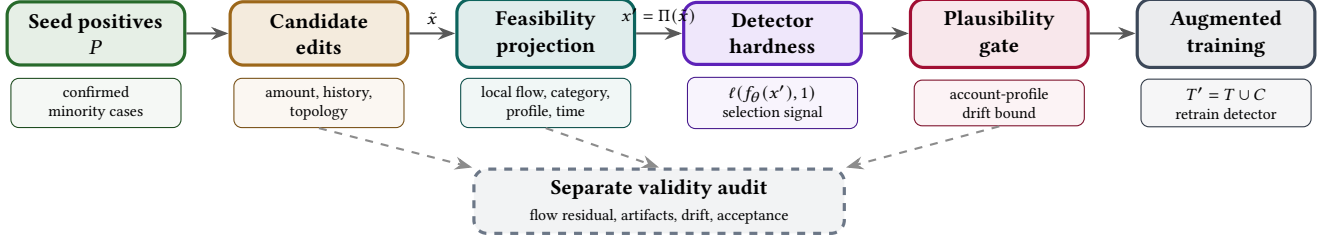


Figure 1: Two-column schematic of FC-CGA. Candidate edits are intentionally local and detector-driven, but only become training examples after feasibility projection and profile plausibility gating. The audit below the pipeline is not a training loss; it is a separate measurement of whether generated examples remain usable as financial transaction records.

Algorithm 2 FC-CGA CounterfactualGenerate

Require: Positive seed set P ; detector f_θ ; augmentation ratio ρ ; candidate-pool multiplier M ; typology τ ; plausibility threshold τ_p ; account statistics $\{\mu_o, \sigma_o\}$

Ensure: Augmentation set C with $|C| \leq \lfloor \rho|P| \rfloor$

- 1: $Q \leftarrow \emptyset$; $n_{\text{aug}} \leftarrow \lfloor \rho|P| \rfloor$
- 2: **for** each $x \in P$, repeated M times **do**
- 3: $\tilde{x} \leftarrow \text{Perturb}(x, w^{(\tau)})$ ▷ typology-weighted edit
- 4: $x' \leftarrow \text{FeasibilityProject}(\tilde{x})$ ▷ Algorithm 1
- 5: $h \leftarrow \ell(f_\theta(x'), 1)$ ▷ detector-loss hardness
- 6: $S_p(x') \leftarrow \exp(-\lambda \text{drift}(x', \mu_o))$ ▷ plausibility, Eq. 5
- 7: **if** $S_p(x') > \tau_p$ and x' passes validity audit **then**
- 8: $Q \leftarrow Q \cup \{(x', h)\}$
- 9: **end if**
- 10: **end for**
- 11: Sort Q by h in descending order (or by curriculum schedule)
- 12: $C \leftarrow \text{top-}n_{\text{aug}}$ items of Q
- 13: **return** C

– $\log(\tau)/\lambda$. Plausibility gating reduces behavioural implausibility. The validity audit in Table 2 shows the intended tradeoff: projected variants remain hard for the detector, while plausibility-aware selection lowers account-profile drift.

4.5 Typology-Aware Generation and Curriculum

Typology-projected variant. The typology-projected variant adapts feature-perturbation weights $w^{(\text{typ})} \in \mathbb{R}^d$ according to the target typology. For layering, weights are elevated for multi-hop flow features and chain-length statistics that characterise funds passing through multiple intermediary accounts. Generation proceeds using the weighted perturbation, then applies the full feasibility projection.

Curriculum projected variant. Rather than selecting only maximally hard examples, the curriculum variant schedules hardness monotonically across epochs: early rounds favour moderate-hardness examples (hardness score below the 50th percentile of the current candidate pool); later rounds shift towards the top-hardness region. This reduces catastrophic boundary distortion in early training, particularly for typologies (e.g. structuring) where the positive-class boundary is subtle and high-hardness examples from the full positive set are unrepresentative.

Table 1: Transaction-graph benchmarks. The validity audit is only fully applicable on datasets that expose local-flow and account-profile fields.

Dataset	#Tx	#Pos	Typology labels	Full audit
AMLNet	1.09M	1,728	yes	yes
TransXion	3.01M	4,631	no	yes
Elliptic++	0.20M	4,545	no	no

5 EXPERIMENTAL SETUP

The experimental design follows four questions: (i) do common augmentations produce transaction records that pass a financial-validity audit; (ii) does post-hoc repair of the failing artifacts match validity-aware counterfactual generation; (iii) does valid hard support actually improve ranking and alert-budget metrics under a strict emerging-typology protocol; and (iv) where does the method fail, and what do those failures say about the geometry of the target typology?

Datasets. We use three transaction-graph benchmarks (Table 1). AMLNet is the primary benchmark because it contains typology labels for layering, structuring, and integration, which make held-out typology transfer possible. TransXion is larger and exposes richer account-profile features; we use it as a full-data robustness check rather than for typology transfer because it has no public typology labels. Elliptic++ serves as an external predictive sanity check; we do not run the full validity audit on it because its public feature schema does not expose the local-flow and profile fields the audit requires.

Typology-transfer protocol. For held-out typology transfer, all positives of the target typology are removed from the chronological training split. The detector is trained on the remaining stream; counterfactuals are generated only from positives available in that training split; and evaluation is restricted to future positives of the held-out typology. Layering is the primary positive setting because it has enough support for ten-seed bootstrap evaluation and the local-flow counterfactual family matches its dominant signal. Structuring and integration follow the same protocol as boundary-condition tests.

Detectors and baselines. The headline detectors are LightGBM [16] and XGBoost [4], which are strong, stable, and widely deployed on tabular transaction features. We also include PyG GraphSAGE [14] and GAT [30] as graph-model checks, reported as limitations rather than secondary wins. Baselines are grouped by purpose: *no augmentation* measures the detector alone; *feature*

noise, SMOTE [3], and mixup [39] stand in for standard feature-space augmentation; their *repaired* variants remove audited surface artifacts post-hoc; *random feasible* augmentation isolates validity without hardness; and *unprojected adversarial* augmentation is a validity stress test rather than a competitive baseline. Our pipeline yields three variants: *hard projected*, *curriculum projected*, and *typology projected*, each with an optional *plausible* sub-variant that adds profile-drift gating.

Training protocol. Each seed runs the same loop: train the base detector on the typology-held-out split; generate candidates from the available positives; repair or project them according to the method; select candidates by detector hardness when applicable; and retrain on the augmented set. Final transfer runs use augmentation ratio $\rho=0.75$, a candidate pool $8\times$ larger than the selected set, log-normal amount perturbation capped at the 99.5th seed-amount percentile, and method-specific log-normal scales tuned once on a validity-audit grid and frozen for all reported runs. Candidate selection requires normalized flow residual ≤ 0.03 . Plausible variants additionally keep candidates below the 60th profile-drift percentile and the 80th edit-distance percentile of the candidate pool; the curriculum variant uses a 90th-percentile profile gate and schedules hardness from moderate to high-loss candidates across epochs.

Metrics and reporting. The primary predictive metric is AUPRC [9], which is appropriate for severely imbalanced detection. Because AML systems operate under analyst capacity, we additionally report low-FPR recall and top-alert precision. *Validity* is evaluated on a separate axis through artifact rates, audited flow residual, profile drift, candidate acceptance, and detector hardness. The main held-out layering experiment uses ten random seeds and a prediction-level paired bootstrap with 200 resamples per seed; these seeds capture detector and augmentation stochasticity on a fixed chronological split. All comparisons are paired: for a fixed detector and seed, the same chronological split, negative population, and evaluation positives are reused across methods, so the relevant quantity is the change induced by augmentation rather than absolute AUPRC. We treat invalid-but-predictive, valid-but-uninformative, and valid-and-hard augmentations as three distinct outcomes, and require the proposed methods to improve against both random-feasible and repaired feature-space controls before reporting a positive result.

Implementation and reproducibility. Boosted-tree detectors are trained with 1,000 rounds, learning rate 0.05, max depth 8, and class-weighted loss (positive weight set to the inverse positive rate of the training split). For PyG GraphSAGE and GAT we use two layers, 64 hidden units, dropout 0.3, Adam at $\text{lr } 10^{-3}$, and neighbor sampling with fan-out (20, 10); training is run for 100 epochs with early stopping on a held-in validation slice. Counterfactual generation samples $M=8$ candidates per positive, applies the projection in Algorithm 1, and computes the hardness score with a single forward pass through the trained base detector. The profile drift used in plausibility gating is $\|x'_h - \mu_v\|_2 / \|\sigma_v\|_2$ averaged across account-history dimensions, with λ in Eq. 5 fixed to 1.0. Audited flow residual is $|\Delta_{\text{out}}(u) - \Delta_{\text{in}}(v)|$ on the edited transaction. All experiments run on a single CPU node with 96 GB RAM; the entire ten-seed held-out layering grid (no-augmentation plus all augmentation variants, both detectors, all stress slices) completes in under 9 wall-clock hours.

Table 2: Validity audit on generated examples. Invalidity reports nonzero audited failure signals; “valid” means no audited surface artifact. Feature noise and unprojected adversarial augmentation are stress tests, while repaired and random-feasible rows are the meaningful controls.

Method family	Invalidity signal	Hard.	Drift
Feature noise	58.2 L / 49.9 C / 35.2 N	2.43	2.08
Unprojected adversarial	96.9 L / 0.461 residual	2.60	0.68
SMOTE + repair	valid	0.27	1.46
Mixup + repair	valid	1.48	1.08
Random feasible	valid	0.03	1.30
Hard projected	valid	6.21	0.53
Typology projected	valid	7.05	0.52
Plausible typology	valid	5.71	0.29

L: local-flow violation; C: categorical artifact; N: negative feature.

6 RESULTS

Section 6.1 reports the validity audit and shows that predictive hardness and financial validity are nearly decorrelated across augmentation families. The held-out layering result is reported as a paired comparison (§6.2), evaluated at operational alert budgets (§6.4), and stress-tested under few-shot, temporal, and new-counterparty slices (§6.5). Boundary conditions on non-layering typologies and graph neural detectors are reported in §6.6.

6.1 Validity Is A Separate Evaluation Axis

Table 2 reports predictive hardness and financial validity as separate measurements. The two axes are nearly decorrelated across augmentation families. Feature noise violates multiple constraints simultaneously (58.2% local-flow violations, 49.9% categorical artifacts, 35.2% impossible negative values); unprojected adversarial perturbations are nominally hard but achieve their hardness by leaving the feasible transaction space (96.9% local-flow violation rate). Post-hoc repair removes audited surface artifacts but collapses detector hardness: repaired SMOTE drops to a hardness of 0.27, indistinguishable from no augmentation. Random feasible augmentation passes the audit but is almost trivial for the detector (0.03 hardness). Only the projected variants occupy the region the pipeline targets: audited validity at hardness 6–7, with substantially lower profile drift once plausibility gating is added.

An augmentation method that improves AUPRC while failing the audit is detecting its own artifacts rather than future positives, and a method that passes the audit while producing uninformative examples cannot be expected to improve detection under emerging-typology shift. The projected family is the only one that satisfies both conditions, and the predictive comparisons that follow are evaluated against repaired and random-feasible controls.

A separate observation worth noting is that plausibility gating reduces profile drift from 0.53 (hard projected) to 0.29 (plausible typology) without proportionally sacrificing hardness (6.21 $\rightarrow$ 5.71). This is the design target of §4.4: gating trades a small amount of hardness for a substantial reduction in account-profile deviation. In deployment terms, a generated transaction whose profile statistics deviate strongly from the source account is exactly the kind of

Table 3: Main held-out layering result. Values are mean $\pm$ std over ten seeds. The control cell reports the best repaired standard augmentation and random feasible augmentation; the evidence cell reports the paired interval, seed wins, and alert-budget gains of the best projected method over no augmentation.

Det.	No aug.	Controls	Best projected	Paired evidence
LGB	0.932 $\pm$ 0.001	repair 0.934 $\pm$ 0.005 feasible 0.934 $\pm$ 0.002	0.947 $\pm$ 0.002 Typology proj. $\Delta = +0.016$	[+0.007, +0.025] 10–0 wins R@0.1 +0.017, P@500 +0.013
XGBoost	0.927 $\pm$ 0.001	repair 0.933 $\pm$ 0.005 feasible 0.932 $\pm$ 0.002	0.946 $\pm$ 0.003 Curriculum proj. $\Delta = +0.018$	[+0.010, +0.028] 10–0 wins R@0.1 +0.020, P@500 +0.015

artifact a human reviewer would flag as implausible even before the detector scored it, so reducing drift while retaining hardness is desirable on both predictive and operational grounds.

6.2 Held-Out Layering Transfer

Table 3 reports the held-out layering result for four comparisons: no augmentation, the best repaired feature-space baseline, random feasible augmentation, and the best projected variant. For LightGBM, the best projected variant lifts AUPRC from 0.932 to 0.947 (paired bootstrap [+0.007, +0.025], 10/10 seed wins). For XGBoost, the lift is from 0.927 to 0.946 (paired bootstrap [+0.010, +0.028], 10/10 seed wins). Repaired SMOTE and random feasible augmentation move the mean by at most +0.005, with bootstrap intervals that include zero. The gain is not isolated to AUPRC; it transfers to operational metrics, with $R@0.1\%$ FPR rising by 0.017–0.020 and Precision@500 by 0.013–0.015.

Figure 2 plots the per-seed AUPRC gain over no augmentation. Random feasible augmentation gives the smallest deltas; repaired feature-space augmentation barely separates from the baseline; the largest and most stable deltas come from projected hard, curriculum, and typology variants. Feasibility alone is therefore not predictive of utility: the useful signal comes from counterfactuals that are simultaneously *feasible* and close enough to the held-out layering boundary to apply real pressure on the detector.

All comparisons are paired against the same no-augmentation detector and the same evaluation positives, so the relevant quantity is the change induced by augmentation rather than absolute AUPRC. Across detectors, the same effect holds: feasibility produces usable records, while hardness selection is what moves the future-positive ranking.

6.3 Paired Bootstrap Significance

Because the absolute AUPRC values on held-out layering are numerically close, we assess statistical significance with a prediction-level paired bootstrap. For each detector, seed, and method pair, we resample the test predictions (positives and negatives) with replacement 200 times and compute the Δ AUPRC distribution. The 95% intervals are then summarized across the ten seeds. Table 4 reports the means.

Three observations follow. First, every projected variant has a strictly positive bootstrap lower bound on both detectors. Second, SMOTE+repair has a bootstrap interval that includes zero for LightGBM, confirming that the visible mean gain is not statistically

Table 4: Paired bootstrap intervals on held-out layering (Δ AUPRC), 200 resamples per seed, mean across ten seeds. “Wins” is the fraction of seeds where the lower bound exceeds zero.

Det.	Method vs. no aug.	Mean Δ	Bootstrap CI	Wins
LGB	SMOTE + repair	+0.000	[−0.003, +0.003]	0.6
LGB	Random feasible	+0.002	[−0.001, +0.006]	1.0
LGB	Hard projected	+0.014	[+0.005, +0.024]	1.0
LGB	Curriculum projected	+0.013	[+0.006, +0.023]	1.0
LGB	Typology projected	+0.016	[+0.007, +0.025]	1.0
LGB	Plausible typology	+0.014	[+0.006, +0.024]	1.0
XGB	SMOTE + repair	+0.002	[−0.000, +0.005]	1.0
XGB	Random feasible	+0.004	[+0.001, +0.008]	1.0
XGB	Hard projected	+0.018	[+0.008, +0.028]	1.0
XGB	Curriculum projected	+0.018	[+0.010, +0.028]	1.0
XGB	Typology projected	+0.014	[+0.004, +0.026]	1.0
XGB	Plausible typology	+0.013	[+0.002, +0.025]	1.0

distinguishable from no augmentation. Third, the projected family separates from random feasible augmentation (roughly 0.012–0.014 larger Δ , with non-overlapping intervals on LightGBM), so the improvement cannot be attributed to feasibility alone.

6.4 Operational Alert Budgets

AML systems are not deployed by sorting an entire benchmark; analysts work through a capped queue of alerts per period. The paired evidence in Table 3 therefore reports changes at low false-positive and top-alert operating points rather than introducing another full metric table. The best projected variant raises $R@0.1\%$ FPR by +0.017 (LightGBM) and +0.020 (XGBoost), and Precision@500 by +0.013 and +0.015 respectively. Precision@100 is omitted because it saturates near 1.0 across nearly all methods and does not discriminate. Translated to capacity: at a fixed 0.1% FPR alert budget, the projected variant brings into the alert window roughly two additional held-out layering positives per hundred actually present compared to the no-augmentation baseline—a sizeable shift at the scale typical of bank-side AML triage.

6.5 Stress Tests: Few-Shot, Time, and New Counterparties

Table 5 reports three deployment-relevant slices. (i) *Few-shot adaptation*. Adding ten confirmed target-typology positives to the training set narrows the gap but does not eliminate it (Δ =+0.013/+0.012 for LightGBM/XGBoost): projected augmentation remains useful after a small number of real positives arrive. (ii) *Temporal drift*. Bucketing the test stream into time quartiles shows the largest gain in the earliest future bucket (Δ =+0.083/+0.080), where the no-augmentation baseline is weakest. This bucket is the closest analogue to a fresh-deployment regime, in which an emerging typology has begun appearing but training data has not yet been refreshed. (iii) *New counterparties*. Restricting evaluation to future sender-receiver pairs unseen during training isolates the cold-start regime; profile-aware projected variants yield the largest improvements (Δ =+0.023/+0.020). The sparse low-history entity subset contains only 11 positives and is omitted from the main table.

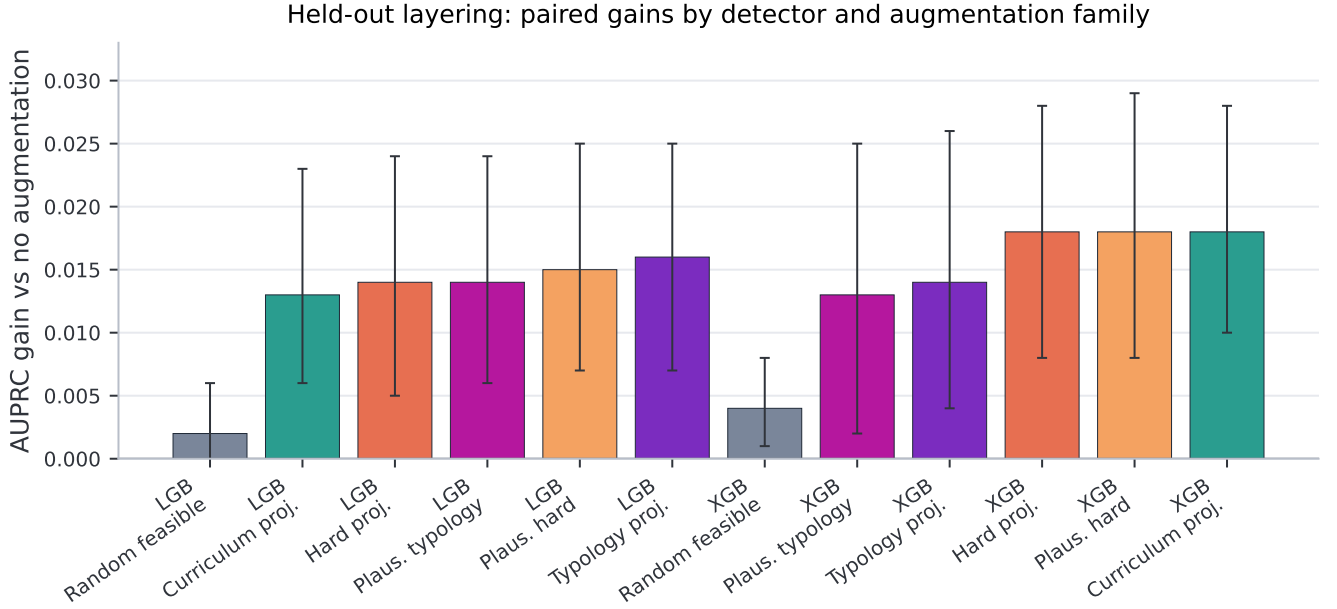


Figure 2: Paired held-out layering AUPRC deltas against no augmentation over ten seeds. Positive values indicate improvement over the corresponding no- augmentation detector; error bars show the prediction-level bootstrap interval summarized across seeds.

Table 5: Stress tests for held-out layering. Values are AUPRC mean $\pm$ std.

Slice	Detector	No aug.	Best projected	Δ
Few-shot	LGB	0.938 $\pm$ 0.003	0.951 $\pm$ 0.005	+0.013
Few-shot	XGB	0.936 $\pm$ 0.005	0.948 $\pm$ 0.002	+0.012
Temporal Q1	LGB	0.655 $\pm$ 0.006	0.738 $\pm$ 0.014	+0.083
Temporal Q1	XGB	0.651 $\pm$ 0.002	0.731 $\pm$ 0.024	+0.080
New pair	LGB	0.928 $\pm$ 0.002	0.951 $\pm$ 0.005	+0.023
New pair	XGB	0.923 $\pm$ 0.001	0.943 $\pm$ 0.009	+0.020

Each slice corresponds to a shift that typically triggers AML model retraining in practice. The gain is concentrated in the regimes where the no-augmentation baseline is weakest—earliest future bucket and unseen counterparties—consistent with the role of projected augmentation as a label-scarcity mechanism rather than an in-distribution oversampler. Figure 3 places these wins alongside the failures of §6.6.

6.6 Boundary Conditions

Figure 3 reports the negative results. Structuring is the cleanest case: projected examples remain valid but move the detector toward the wrong support region, reducing AUPRC from 0.618 $\pm$ 0.041 to 0.390 $\pm$ 0.003 for LightGBM and from 0.625 $\pm$ 0.029 to 0.387 $\pm$ 0.006 for XGBoost. Random feasible augmentation outperforms hard projection on XGBoost structuring, indicating that the failure is not driven by invalid examples but by projected support that no longer matches the held-out structuring region (§7.1). Integration AUPRCs are at the 10^{-4} scale across all methods and seeds; we report only

the relative ordering. The PyG GNN check on held-out layering does not reproduce the boosted-tree improvement: mean AUPRC stays near the baseline with high seed variance (0.705 $\pm$ 0.089 for GAT, 0.355 $\pm$ 0.275 for GraphSAGE), suggesting that the local edge edits do not survive neighborhood aggregation as a clean training signal. FC-CGA is therefore typology-sensitive and detector-sensitive, rather than a universal augmentation recipe.

7 ABLATIONS AND ANALYSIS

7.1 Why Layering Helps But Structuring Fails

The most informative diagnostic comes from comparing generated and held-out examples in feature space. For layering, projection moves generated examples from a mean nearest-neighbor distance of 1.18 (random feasible) to 0.73 (plausible typology) against held-out layering positives, while raising detector hardness from 0.04 to 7.95 (Table 6). The generated log-amounts (≈ 7.0) are close to the held-out target (8.76). For structuring, NN distance to held-out positives drops similarly (from 2.34 to 0.82) and hardness climbs to 8.19, but the generated log-amounts collapse to 5.63 against a target of 8.58. Integration is extreme: generated log-amounts (≈ 5.9) sit nearly five orders of magnitude below the target (10.90).

Geometric proximity is necessary but not sufficient. Hard projected examples are close to held-out positives in standardized feature space, but “close” in standardized coordinates can still mean “far” in the amount regime that defines the typology. For structuring—the canonical small-amount typology—hard projection drives amounts *downward* below even the structuring threshold, so the detector is trained on valid hard examples from the wrong semantic region. This is the cleanest mechanistic explanation we have for the

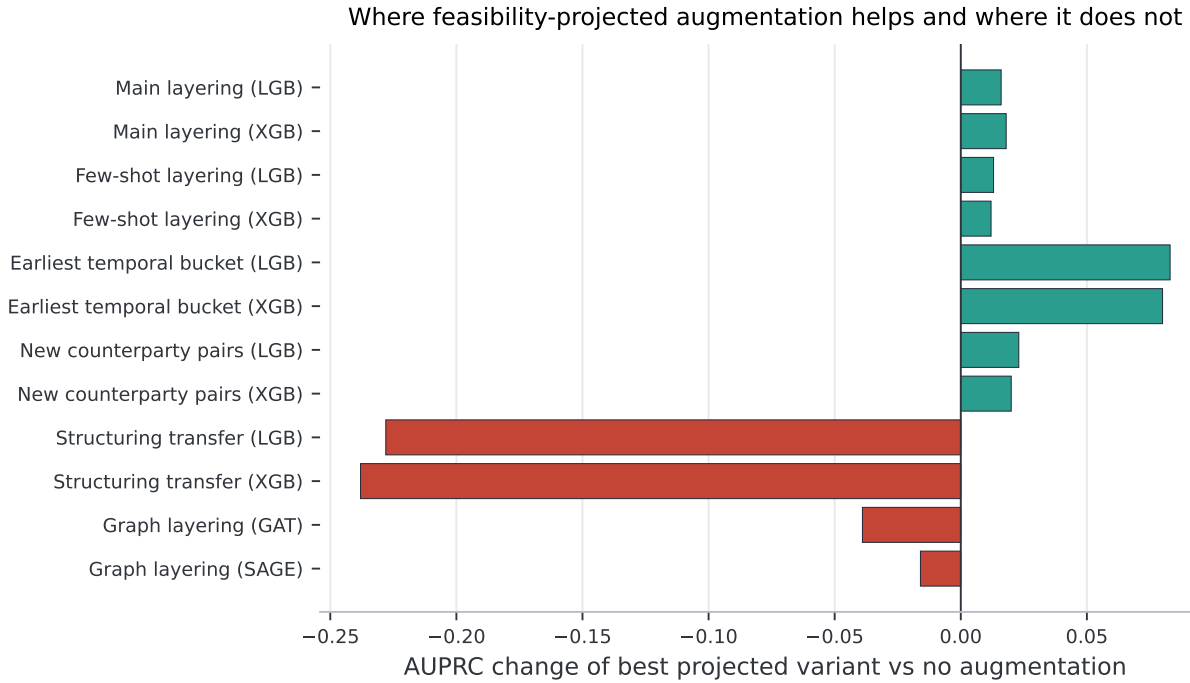


Figure 3: Scope of the empirical effect. Positive bars are layering settings in which the best projected variant improves over no augmentation; negative bars show failure cases where the same family does not transfer.

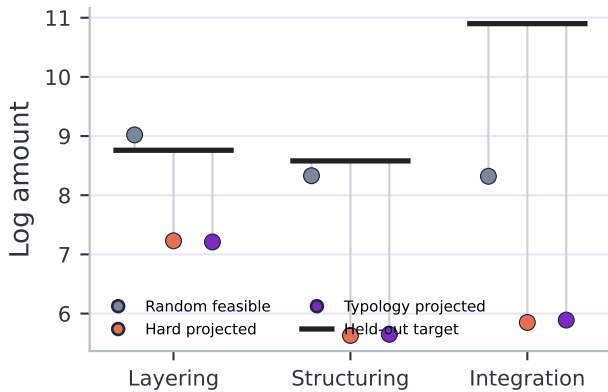


Figure 4: Amount alignment explains the typology boundary. The black segment marks the held-out target amount regime; colored points show generated support. Layering remains close enough to be useful, while projected structuring and integration examples move into much lower amount regimes.

negative result in Figure 3: feasibility-aware augmentation is a *support-coverage* method, not a *semantic-typology* method.

Figure 4 and Table 6 make this precise: for structuring and integration, the generated examples are valid and hard but lie outside the amount regime of the target typology. The practical implication is that any validity-aware augmentation deployment should be accompanied by a typology audit comparing the generated and target amount distributions before committing candidates to training.

Table 6: Mechanism diagnostics on AMLNet generated examples. NN distance is to held-out positives in standardized transaction/history feature space.

Typology	Method	NN dist.	Hard.	Drift	Log amount	
					Generated	Target
Layering	Random feasible	1.182±0.285	0.04	2.55	9.02	8.76
Layering	Hard projected	0.764±0.022	9.11	1.11	7.23	8.76
Layering	Plausible typology	0.726±0.027	7.95	0.85	6.97	8.76
Structuring	Random feasible	2.341±0.210	0.03	1.77	8.33	8.58
Structuring	Hard projected	0.816±0.020	8.19	0.63	5.63	8.58
Structuring	Typology projected	0.833±0.019	8.34	0.63	5.65	8.58
Integration	Random feasible	5.831±0.080	0.01	1.76	8.32	10.90
Integration	Hard projected	6.610±0.023	8.10	0.69	5.85	10.90
Integration	Typology projected	6.607±0.020	8.40	0.70	5.89	10.90

7.2 Constraint Ablations

Table 7 isolates which constraints and perturbation families carry the result. Three patterns emerge. First, on AMLNet the no-flow ablation can raise *raw* AUPRC slightly above the full projection, but this is not a deployable improvement: the audit shows the detector is exploiting amount movement that fails the local flow check. This is precisely the validity gap recovered as an ablation. Second, on TransXion the ordering reverses: profile-removal hurts sharply (e.g. 0.269→0.240 for LightGBM), and topology-only perturbation is best. Third, the useful feasible direction is therefore dataset- and typology-dependent; the validity projection defines the feasible set, but does not decide which feasible directions are semantically useful.

Table 7: Constraint ablation AUPRC (mean $\pm$ std, 3 seeds). Higher raw AUPRC under no-flow should not be read as a valid improvement; it indicates that invalid amount movement can be predictive if the model is allowed to exploit it.

Dataset	Det.	Hard proj.	no-flow	no-profile	amt.-only	topo.-only
AMLNet	LGB	0.977 $\pm$ 0.001	0.980$\pm$0.001	0.978 $\pm$ 0.001	0.974 $\pm$ 0.001	0.971 $\pm$ 0.001
AMLNet	XGB	0.969 $\pm$ 0.001	0.976 $\pm$ 0.002	0.976$\pm$0.002	0.972 $\pm$ 0.001	0.971 $\pm$ 0.001
TransXion	LGB	0.269 $\pm$ 0.003	0.262 $\pm$ 0.003	0.240 $\pm$ 0.001	0.265 $\pm$ 0.001	0.271$\pm$0.002
TransXion	XGB	0.236 $\pm$ 0.004	0.237 $\pm$ 0.006	0.230 $\pm$ 0.002	0.242 $\pm$ 0.006	0.248$\pm$0.005

7.3 Additional Observations

Three further observations are worth noting briefly. (i) *Full-data AMLNet*. When all typologies are visible during training, projected variants still narrowly lead on AMLNet (0.978 vs. 0.971 no-aug for LightGBM, Table omitted), but the margin is small enough that the held-out typology setting remains the more discriminating evaluation. (ii) *TransXion full-data*. TransXion is closer to a no-shift setting and favors lightly repaired feature-space augmentation (e.g. feature-noise+repair at 0.280 for LightGBM, vs. 0.272 no-aug); projected variants do not improve here, consistent with the support-coverage interpretation—there is little untouched future-typology region for them to fill. (iii) *Augmentation ratio*. A sweep over $\rho \in \{0.25, 0.5, 0.75, 1.0\}$ shows the layering result is stable in the $[0.5, 1.0]$ band; we fix $\rho=0.75$ for all reported runs. This rules out a narrowly tuned ratio as the source of the gain.

8 DISCUSSION AND LIMITATIONS

Scope of the result. Validity-aware counterfactual augmentation improves held-out layering transfer on the AMLSim-derived benchmark across low-FPR alert budgets, temporally drifted future data, and previously unseen counterparty pairs, while repaired feature-space and random-feasible baselines do not. The gain is consistent across ten seeds, prediction-level paired bootstrap intervals, two strong boosted-tree detectors, and three deployment-relevant slices. It does not extend to graph neural detectors or to non-layering AML typologies under the same per-edge projection family.

Asymmetric validity–utility tradeoff. Financial validity removes a failure mode of augmentation but does not guarantee utility. The constraint ablation in Table 7 shows that dropping the flow constraint sometimes raises raw AUPRC on the AMLSim-derived dataset because the learner exploits amount movement that fails the audit. As an in-distribution metric this appears as a positive ablation; as a deployment metric it is a warning that augmentation in financial settings should be evaluated on two axes by default, not one.

Why structuring and graph models fail. The mechanism analysis identifies two distinct failure modes. For structuring, the projected support has the right geometric proximity to held-out positives in standardized feature space, but the amount regime collapses below the typology’s defining threshold; the detector receives hard examples from the wrong semantic region. For graph neural detectors, the local edge edits interact with neighborhood aggregation, temporal sampling, and message propagation in ways the per-edge feasibility projection does not control. A useful individual transaction record is not necessarily a useful node or subgraph representation update.

Complexity and overhead. For a positive seed set of size $|P|$ and pool multiplier M , candidate generation requires $O(M|P|)$ forward passes through the detector for the hardness score, plus $O(M|P|d)$ time for projection, where d is the feature dimension. On AMLNet ($|P| \approx 1,700$) with $M=8$ and $\rho=0.75$, FC-CGA adds roughly 12–18 seconds of overhead per seed on top of the LightGBM training time, dominated by detector forward passes. Memory overhead is negligible because candidates are scored in mini-batches and only the top- n_{aug} are retained. The projection itself is closed-form and introduces no extra trainable parameters.

Limitations and future work. Three limitations bound the result. (i) The AMLSim-derived benchmark provides the clean typology labels needed for held-out transfer, but it is synthetic; TransXion is larger and profile-rich but lacks typology annotations. A natural next step is reproducing the held-out protocol on a production-derived dataset with confirmed typology labels. (ii) The feasibility projection is closed-form and per-edge; coupling it with a *typology generator* that reasons over multi-edge episodes, amount schedules, and temporal spacing should narrow the structuring and integration gap. (iii) Differentiable projection layers and projection-aware GNN training objectives are an obvious path to making the same validity mechanism useful for message-passing detectors. Each of these extensions is consistent with the two-axis evaluation framing introduced here rather than a departure from it.

9 CONCLUSION

We argued that augmentation for transaction-graph fraud detection should be evaluated on *two* axes: predictive utility and financial validity. FC-CGA operationalizes this view through a closed-form feasibility projection that enforces local flow balance, categorical integrity, account-profile bounds, and temporal feasibility, paired with detector-hardness selection and profile-drift gating. The evidence is sharpest on held-out layering transfer: across ten seeds, two boosted-tree detectors, paired bootstrap intervals, and three deployment-relevant slices, valid projected counterfactuals improve both AUPRC and operational alert-budget metrics over no augmentation, repaired SMOTE/mixup, and random-feasible controls. The same family *does* not improve structuring, integration, or graph neural detectors, and we explain those failures through a mechanism analysis showing that geometric proximity in standardized feature space is necessary but not sufficient—the generated support must also match the semantic geometry of the target typology. Taken together, the results argue for typology-sensitive, validity-audited augmentation as a more credible path for financial fraud detection than unconstrained synthetic generation.

REFERENCES

- [1] Edoardo Altman, Jovan Blanuša, Luc Von Däniken, Philip Iff, Michal Kaitchuck, Ilaria Bordino, Marco Marini, and Mattia Maggioni. 2023. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, 44641–44654.
- [2] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural Temporal Graph Neural Networks for Anomaly Detection in Dynamic Graphs. In *ACM International Conference on Information and Knowledge Management (CIKM)*. 3747–3756.
- [3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357.
- [4] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on*

- Knowledge Discovery and Data Mining*. ACM, San Francisco, CA, USA, 785–794.
- [5] Dawei Cheng, Xiaoyang Wang, Ying Zhang, and Liqing Zhang. 2022. Graph Neural Network for Fraud Detection via Spatial-Temporal Attention. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2022), 3800–3813.
- [6] Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V. Le. 2019. AutoAugment: Learning Augmentation Strategies from Data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Long Beach, CA, USA, 113–123.
- [7] Damien Dablain, Bartosz Krawczyk, and Nitesh V. Chawla. 2023. DeepSMOTE: Fusing Deep Learning and SMOTE for Imbalanced Data. *IEEE Transactions on Neural Networks and Learning Systems* 34, 9, 6390–6404.
- [8] Andrea Dal Pozzolo, Olivier Caelen, Yann-Aël Le Borgne, Serge Waterschoot, and Gianluca Bontempi. 2014. Learned Lessons in Credit Card Fraud Detection from a Practitioner Perspective. *Expert Systems with Applications* 41, 10 (2014), 4915–4928.
- [9] Jesse Davis and Mark Goadrich. 2006. The Relationship Between Precision-Recall and ROC Curves. In *Proceedings of the 23rd International Conference on Machine Learning*. ACM, Pittsburgh, PA, USA, 233–240.
- [10] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S. Yu. 2020. Enhancing Graph Neural Network-based Fraud Detection via Balanced and Augmented Graph Learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. ACM, Virtual Event, Ireland, 315–324.
- [11] Mina Elmougy, Siyuan Liu, and Jiawei Han. 2023. Elliptic++: A Graph Network of 803M Transactions and 122M Addresses in the Bitcoin Blockchain. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, Long Beach, CA, USA, 4168–4178.
- [12] João Gama, André Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4 (2014), 1–37.
- [13] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *Proceedings of the 3rd International Conference on Learning Representations*. San Diego, CA, USA.
- [14] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, 1024–1034.
- [15] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive Multi-View Representation Learning on Graphs. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 4116–4126.
- [16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, 3146–3154.
- [17] Kezhi Kong, Jiuhai Chen, John Kirchenbauer, Renkun Ni, Micah Goldblum, and Tom Goldstein. 2022. FLAG: Adversarial Data Augmentation for Graph Neural Networks. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence*. AAAI Press, 7461–7469.
- [18] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2021. Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection. In *Proceedings of the Web Conference 2021*. ACM, Ljubljana, Slovenia, 3168–3177.
- [19] Ana Lucic, Maartje A. ter Hoeve, Gabriele Tolomei, Maarten de Rijke, and Fabrizio Silvestri. 2022. CF-GNNExplainer: Counterfactual Explanations for Graph Neural Networks. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 151)*. PMLR, 4499–4511.
- [20] Jing Ma, Ruocheng Guo, Saumitra Mishra, Aidong Zhang, and Jundong Li. 2022. CLEAR: Generative Counterfactual Explanations on Graphs. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, 25895–25907.
- [21] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *Proceedings of the 6th International Conference on Learning Representations*. Vancouver, BC, Canada.
- [22] Divyat Mahajan, Chenhao Tan, and Amit Sharma. 2019. Preserving Causal Constraints in Counterfactual Explanations for Machine Learning Classifiers. In *Proceedings of the NeurIPS Workshop on Do the Right Thing: Machine Learning and Causal Inference for Improved Decision Making*.
- [23] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, and Charles E. Leiserson. 2020. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*. AAAI Press, New York, NY, USA, 5363–5370.
- [24] Joonhyung Park, Jaeyun Song, and Eunho Yang. 2022. GraphENS: Neighbor-Aware Ego Network Synthesis for Class-Imbalanced Node Classification. In *Proceedings of the 10th International Conference on Learning Representations*.
- [25] Tahereh Pourhabibi, Kok-Leong Ong, Booi H. Kam, and Yee Ling Boo. 2020. Fraud Detection: A Systematic Literature Review of Graph-Based Anomaly Detection Approaches. *Decision Support Systems* 133 (2020), 113303.
- [26] Rafael Poyiadzi, Kacper Sokol, Raul Santos-Rodriguez, Tijl De Bie, and Peter Flach. 2020. FACE: Feasible and Actionable Counterfactual Explanations. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. ACM, New York, NY, USA, 344–350.
- [27] Liang Qu, Huaisheng Zhu, Ruiqi Zheng, Yuhui Shi, and Hongzhi Yin. 2021. IMGAGN: Imbalanced Network Embedding via Generative Adversarial Graph Networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1390–1398.
- [28] Toyotaro Suzumura and Hiroki Kanezashi. 2019. Towards Scalable and Privacy-Preserving Graph Neural Network-Based Fraud Detection for Financial Transactions. In *IEEE International Conference on Big Data*. IEEE, 3184–3193.
- [29] Toyotaro Suzumura and Hiroki Kanezashi. 2021. AMLSim: Anti-Money Laundering Transaction Simulator. GitHub repository, IBM Research. <https://github.com/IBM/AMLSim>.
- [30] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations*. Vancouver, BC, Canada.
- [31] Sahil Verma, John Dickerson, and Keegan Hines. 2020. Counterfactual Explanations for Machine Learning: A Review. *arXiv preprint arXiv:2010.10596* (2020).
- [32] Sandra Wachter, Brent Mittelstadt, and Chris Russell. 2017. Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR. *Harvard Journal of Law & Technology* 31, 2 (2017), 841–887.
- [33] Mark Weber, Jie Chen, Toyotaro Suzumura, Aldo Pareja, Tengfei Ma, Hiroki Kanezashi, Charles E. Leiserson, and Andrei Lapets. 2018. Scalable Graph Learning for Anti-Money Laundering: A First Look. *arXiv preprint arXiv:1812.00076* (2018).
- [34] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl Ingo Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. 2019. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. In *Proceedings of the KDD Workshop on Anomaly Detection in Finance*. Anchorage, AK, USA.
- [35] Lirong Wu, Jun Xia, Zhangyang Gao, Haitao Lin, Cheng Tan, and Stan Z. Li. 2022. GraphMixup: Improving Class-Imbalanced Node Classification by Reinforcement Mixup and Self-Supervised Context Prediction. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 519–535.
- [36] Bingbing Xu, Huawei Shen, Bingjie Sun, Rong An, Qi Cao, and Xueqi Cheng. 2021. Towards Consumer Loan Fraud Detection: Graph Neural Networks with Role-Constrained Conditional Random Field. In *AAAI Conference on Artificial Intelligence*. 4537–4545.
- [37] Yuning You, Tianlong Chen, Yonglong Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, 5812–5823.
- [38] Sangdo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. 2019. CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. IEEE, Seoul, South Korea, 6023–6032.
- [39] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. In *Proceedings of the 6th International Conference on Learning Representations*. Vancouver, BC, Canada.
- [40] Tianxiang Zhao, Gang Liu, Daheng Wang, Wenhao Yu, and Meng Jiang. 2022. Counterfactual Graph Learning for Link Prediction. In *International Conference on Machine Learning (ICML)*.
- [41] Tianxiang Zhao, Xianfeng Zhang, and Suhang Wang. 2021. GraphSMOTE: Imbalanced Node Classification on Graphs with Graph Neural Networks. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. ACM, Virtual Event, Israel, 833–841.
- [42] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep Graph Contrastive Representation Learning. In *Proceedings of the ICML Workshop on Graph Representation Learning and Beyond*.